SP-4 Phase G — Removal-syncs-to-backup (detailed design)

Spec date: 2026-05-13 **Parent SP:** `docs/superpowers/specs/2026-05-12-sp4-multi-provider-redesign.md` **Prerequisite:** Phases A + B + C + D + F.1 complete (HEAD `2cf53766`). **Status:** detailed-design locked. Implementation plan in `docs/superpowers/plans/2026-05-13-sp4-phase-g-implementation.md`.

Goal (from parent SP-4 design)

> **Phase G — Removal-syncs-to-backup.** Soft-delete with sync-up semantics so a removal propagates to other devices and survives reinstall via backup. Backup-restore test on a second emulator.

Why soft-delete

Today's `delete()` is a hard SQL DELETE — the row vanishes. Two failure modes:

1. **Cross-device propagation impossibility.** Phase E's sync mechanism (owed) needs to upload a "removal" event to the API. With a hard delete, there's nothing left to upload — the local state has lost the fact that a removal happened.
2. **Backup-restore reanimation.** When a device restores from cloud backup (a snapshot taken before the removal), the deleted row comes back. The user sees deleted credentials reappear after factory reset.

Soft-delete fixes both: the row stays in the database with a `deletedAt` timestamp; read paths filter it out (so the user sees it as deleted); the sync layer can upload the removal event; backup snapshots include the soft-delete marker.

Scope of Phase G

This phase ships the **local half** — soft-delete + outbox enqueue. The remote propagation (Phase E) reads the outbox events and uploads them. Phase G makes Phase E's work possible.

In-scope:

- `credentials_entry` table gets a `deletedAt: Long?` column.
- `cloned_provider` table gets a `deletedAt: Long?` column.
- `provider_credential_binding` already has `unbind(providerId)` which physically deletes the row — for Phase G's semantics this is equivalent (the binding has no "soft-delete needed" property; an unbind followed by a re-bind is the natural pattern).
- All read paths (`observeAll`, `get`, `getAll`) filter `WHERE deletedAt IS NULL`.
- All write paths (CredentialsManager UI delete, ProviderConfig Clone-remove) call `softDelete(id, now)` AND enqueue an outbox event with `deleted = true`.
- The outbox `SyncOutboxKind.CREDENTIALS` and `CLONED_PROVIDER` wire payloads gain a `deleted: Boolean = false` field — existing entries default to `false`, new soft-deletes set it to `true`.
- Room schema bumps v9 → v10 with `MIGRATION_9_10`.

Out of scope (Phase E):

- The actual upload of the soft-delete outbox event to the API.
- The download of remote soft-deletes and their merge into the local DAO.
- Cross-device end-to-end backup-restore Challenge Test on a second emulator.

Design decisions (locked)

G-D1 — `deletedAt` column, not `isDeleted` flag

Storing the deletion timestamp lets Phase E:

- order multiple deletions by when they happened,
- support a future hard-purge sweep (rows where `deletedAt < now − 90 days` are physically deleted to bound DB growth),
- distinguish "deleted N seconds ago" (still in outbox queue waiting to upload) from "deleted N hours ago" (already uploaded; the row is kept as a tombstone).

G-D2 — Two tables get the column

Only `credentials_entry` and `cloned_provider` have user-removable rows that need cross-device propagation. `user_mirror` is already removable via its own DAO + the existing outbox `removed: true` wire flag — no schema change needed. `provider_credential_binding` and `provider_sync_toggle` are upsert-only; the user "removes" a binding by unbinding (which is a write of "unbound" state, not a removal of the row's existence).

G-D3 — Wire payload extension

`CredentialsEntryRepository.upsert(entry)` and `delete(id)` already enqueue `SyncOutboxKind.CREDENTIALS` events. The wire JSON gains a `deleted: Boolean` field that defaults to `false` for upserts and is `true` for soft-deletes. Phase E's API handler distinguishes the two and routes accordingly.

`CloneProviderUseCase` enqueues `SyncOutboxKind.CLONED_PROVIDER` on creation; a new `RemoveClonedProviderUseCase` (added in this phase) enqueues the same kind with `deleted: true`.

G-D4 — Backup-exclusion audit unchanged

Per the SP-4 design's "Operator's invariants reasserted" — the credentials data is at-rest-encrypted, and the soft-delete marker is non-secret metadata. The existing Android Auto Backup configuration (which excludes the encrypted prefs files) does NOT need re-auditing for Phase G; the soft-delete timestamp is included in the backup just like any other Room row.

Affected surfaces

Files to modify

- `core/database/src/main/kotlin/lava/database/entity/CredentialsEntryEntity.kt` — add `deletedAt: Long? = null`.
- `core/database/src/main/kotlin/lava/database/entity/ClonedProviderEntity.kt` — add `deletedAt: Long? = null`.
- `core/database/src/main/kotlin/lava/database/dao/CredentialsEntryDao.kt` — add `softDelete(id, deletedAt)`; filter all reads on `WHERE deletedAt IS NULL`.
- `core/database/src/main/kotlin/lava/database/dao/ClonedProviderDao.kt` — same shape.
- `core/database/src/main/kotlin/lava/database/AppDatabase.kt` — bump `version = 10`; add `MIGRATION_9_10`.

- `core/credentials/src/main/kotlin/lava/credentials/CredentialsEntryRepository.kt` + `Impl.kt` — `delete(id)` routes through `softDelete` + outbox enqueue with `deleted = true`.
- `core/domain/src/main/kotlin/lava/domain/usecase/CloneProviderUseCase.kt` — wire `deleted = false` default in the existing payload; add sibling `RemoveClonedProviderUseCase`.
- `feature/credentials_manager/src/main/kotlin/lava/credentials/manager/CredentialsManagerViewModel.kt` — `Delete` action already calls `repo.delete(id)` — no UI change.

Files to add

- `core/domain/src/main/kotlin/lava/domain/usecase/RemoveClonedProviderUseCase.kt`.
- `core/credentials/src/test/kotlin/lava/credentials/CredentialsEntryRepositorySoftDeleteTest.kt` — verifies soft-deleted row is hidden from `observe()` AND that the outbox carries a `deleted: true` entry.
- `core/database/schemas/lava.database.AppDatabase/10.json` — Room schema export (auto-generated by gradle).

Tests

- **Unit (this phase):** `CredentialsEntryRepositorySoftDeleteTest` —
  1. Upsert a credential. Observe shows it.
  2. Delete the credential. Observe excludes it. Outbox has a `CREDENTIALS` row with `deleted: true`.
  3. Get-by-id returns null (not the soft-deleted row).
  4. Mutation rehearsal: revert `delete` to call `dao.delete()` instead of `dao.softDelete()`. Outbox no longer has the `deleted: true` entry; observe still excludes (because the row is physically gone) — but the upload signal is lost. The test catches the outbox-absence assertion.
- **Challenge (Phase E owed):** cross-device backup-restore on a second emulator. The Phase E design covers this.

Anti-bluff posture (§6.J)

1. Soft-delete is observable by inspecting the Room row directly: a deleted row exists in the table with `deletedAt != null`. The test asserts this directly.
2. The outbox-enqueue assertion checks the wire payload's `deleted` flag — not the count of outbox rows. Verify-only mutations would be caught.

3. The DAO read-path filter is asserted via the `observe()` flow.
4. Falsifiability rehearsal protocol embedded in test KDoc.

Implementation plan

See `docs/superpowers/plans/2026-05-13-sp4-phase-g-implementation.md` — 5 tasks, ~15 steps.